Open Access | https://doi.org/10.25080/ectn7568



SciPy 2025

July 7 - July 13, 2025

Proceedings of the 24th Python in Science Conference
ISSN: 2575-9752

An Active Learning Plugin in napari to Fine-Tune Models for Large-scale Bioimage Analysis

Fernando Cervantes-Sanchez[1]

[1]The Jackson Laboratory

Abstract

The "napari-activelearning" plugin provides a framework to fine-tune deep learning models for large-scale bioimage analysis, such as digital pathology Whole Slide Images. The development of this plugin was motivated by easing the integration of deep learning tools into bioimage analysis workflows. This plugin implements the concept of Active Learning for reducing the time spent on labeling samples when fine-tuning models. Because this plugin is integrated into napari and leverages the use of Next Generation File Formats (e.g. Zarr), it is suitable for fine-tuning deep learning models on large-scale images with little image preparation.

Keywords deep learning, active learning, napari

1. Introduction

Adoption of deep learning methods for bioimage analysis has grown exponentially in recent years. Part of such success is thanks to transfer learning that enables using models that were trained on large volumes of data from diverse domains, such as the ImageNet [1] and Segment Anything 1 Billion (SA-1B) [2] datasets, into tasks where annotated data is scarce. An example is cell segmentation in biological microscopy images [3], which requires human annotation of cell structures in images of different modalities and scales. Such images can be up to tens or hundreds of thousands of pixels per side (e.g. WSI used in histopathology), depending on the acquisition magnification and imaging modality. Moreover, the research community has made considerable efforts during the last years to curate databases for training models relevant to the bioimage analysis field. These databases include LIVECell [4], TissueNet [3], and CellSeg [5], among many others. Such databases can be used for de-novo training or be used for fine-tuning models that were trained with image datasets from general domains.

1.1. Related work

Segmentation of structures in biological image data is a recurrent task in bioimage analysis that serves as an intermediate step for downstream applications. Some of the most common downstream applications for cell segmentation involve cell type classification, counting specific types of cells, and measuring morphological properties of cells in an image. These applications require high quality cell segmentations to obtain accurate information that is used in further bioimage understanding and research. The most relevant deep learning segmentation methods include Cellpose [6], Stardist [7], and Micro-SAM [8]. These methods offer pre-trained models for carrying out segmentation of biological structures in multiple imaging modalities, and tools for fine-tuning those same models to new data with user-defined annotations. Training deep learning models for bioimage analysis involves: 1)

Published Jul 10, 2025

Correspondence to
Fernando Cervantes-Sanchez
fernando.cervantes@jax.org

Open Access

Copyright © 2025 Cervantes-Sanchez. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

extracting several rectangular sections of a specified size (i.e. image tiles) from the original image files, 2) annotating each of those tiles according to the tasks being learned, and 3) storing the pairs of tiles and annotations in separate containers or folders as training and testing datasets. However, this approach involves some technical difficulties such as the costs and time associated with transferring training data between researchers for sharing and reproducibility purposes, costs for storing duplicated data from the image tiles already present in the original images, and lack of context and coordinates from where each tile was originally extracted.

The field of Active Learning studies human-in-the-loop strategies in deep learning that can reduce the time and effort required for de-novo training or fine-tuning models. Applications of active learning have also been developed to address computer vision tasks [9]. This is relevant to biological image analysis where data annotation is one of the most time-consuming tasks [10]. The main reason is that due the scale of the images in some modalities (e.g. WSI), the number of samples that can be extracted for labeling can overwhelm the annotator. On the other hand, for relatively smaller images acquired through other modalities, the annotator would require to annotate large amounts of samples to generate sufficient training data. An active learning sampling strategy, based on acquisition functions, can be used to prevent this by presenting a limited number of samples at a time. Moreover, there are acquisition functions such as BALD [11] that focus on obtaining only cases that could improve a model's performance when correctly labeled.

The bioimage analysis community has been steering towards efficient creation and sharing of repositories containing large volumes of data. NGFFs [12] are one of the most preferred options for large-scale image storage and management, such as the OME-Zarr data format [13]. The Zarr data format establishes a standard structure for compressed, chunked-based, n-dimensional data storage that can be efficiently stored either in local or cloud repositories [13]. Multiple projects have been created on top of Zarr, including APIs such as the zarr-python library [14]. Some image processing softwares have adopted this data format, like Fiji [15] with the MobIE plugin [16], and QuPath [17] with its own Zarr data loader. Visualization tools such as napari [18], neuroglancer [19], viv [20], and webKnossos [21] have also added support to this data format. Additionally, image data stored as Zarr can be used for training deep learning models without duplicating data. That can be achieved by accessing chunked image data instead of extracting image tiles and storing them separately as is commonly needed in standard deep learning training.

In this work, the "napari-activelearning" plugin for napari is introduced as a tool for easing fine-tuning existing models on large-scale images. This plugin leverages napari's user-friendly features for annotating new samples following an active learning workflow. Additionally, the "napari-activelearning" plugin uses NGFF to store annotations in a storage-efficient manner, ready for deep learning training.

2. Methods

The "napari-activelearning" plugin relies on three components to provide a user-friendly framework to train and fine-tune deep learning models for bioimage analysis: 1) NGFF, for efficient storage of the data generated during the active learning workflow, 2) napari, for visualization and graphical interface with the user, and 3) Active Learning, for assisting users with selection of samples of interest that require annotation for improving the deep learning model performance.

2.1. NGFFs

NGFFs [12], such as OME-Zarr [13], have been increasingly adopted by the bioimage analysis community thanks to its computation and storage advantages. Zarr format stores large-

scale image data as independent n-dimensional tiles, also called chunks, either on local disk or cloud storage. By using chunks as units of storage, the amount of data required to be loaded into memory when accessing specific regions of the image is reduced. Accessing image chunks is parallel-safe, which enables acceleration of image processing through parallel computing. This is useful when applying a model for inference in larger-than-memory image data, where deep learning inference can be applied to regions of the image separately, and applying a subsequent reduction operation on the results. This reduction operation could be an accumulation function for whole image classification, or a stitching algorithm for segmentation tasks.

2.2. napari Visualization Tool

napari [18] is a user-friendly n-dimensional data viewer with extensible capabilities via plugins. This visualizer already offers tools for data annotation, and it is compatible with Zarr, enabling visualization of large-scale image data efficiently. napari has been used to develop deep learning applications for bioimaging analysis such as napari-cellpose [6], and Micro-SAM [8] plugins. These plugins can be used to segment biological structures in data acquired with multiple imaging modalities. However, these plugins are intended to be used with tiles of images that have already been extracted and stored, in contrast to the plugin presented in this work, which can be applied on regions defined by the user directly on original images without storing duplicate data from image tiles.

2.3. Active Learning framework

To reduce the amount of data presented to the user for annotation, concepts from the Active Learning framework are implemented in this plugin. This field studies methods for human-in-the-loop learning workflows that avoid overwhelming the annotator with samples to labeling for training. This is achieved through a computation of Acquisition Functions that determine what samples require labeling to subsequently improve the performance of a specific model.

Some of the most used acquisition functions involve Random acquisition, which takes a random unlabeled sample from the dataset with an uniform distribution; Maximize mean standard deviation, that computes the average standard deviation of the probabilities of each class predicted by the model; Maximize variation ratios, which uses the probability of the predicted class of each sample; Maximize the predictive entropy, that is computed from the predicted probabilities for each sample; and Maximize the mutual information between the model parameters and the predicted classes, which is also known as BALD [11]. In this plugin, the BALD [11] acquisition function is implemented to score and sort a set of image patches sampled from an image. This acquisition function was selected because it was demonstrated that can be used efficiently with bioimage data for deep learning training when compared with other functions such as Random acquisition [9]. Such efficiency on selecting the most promising samples is highly desirable for increasing the amount of samples presented to the annotator that could improve the model's performance while reducing redundant samples. In this plugin, the score assigned to a sample extracted at random from the input image is computed following N. Houlsby, F. Huszár, Z. Ghahramani, and M. Lengyel [11]:

$$a(x, M) = \mathbb{I}[y, \theta \mid x, D], \tag{1}$$

where $a(x, M)$ is the acquisition function score for sample x for model M, the mutual information $\mathbb{I}$ is computed from the prediction y made by model M for sample x in the dataset D with the current state of parameters θ. Following Y. Gal, R. Islam, and Z. Ghahramani [9], the mutual information can be computed through Monte Carlo integration to compute the BALD function as follows:

$$\mathbb{I}[y, \theta \mid x, D] = -\sum_{c} \left(\frac{1}{T} \sum_{t} \hat{p_c}^{\,t} \right) log \left(\frac{1}{T} \sum_{t} \hat{p_c}^{\,t} \right) + \frac{1}{T} \sum_{c,t} \hat{p_c}^{\,t} log\, \hat{p_c}^{\,t}, \tag{2}$$

where T is the total number of steps in the Monte Carlo integration, $\hat{p_c}^{\,t}$ is the component of the prediction made by model M for sample x at class c. To apply the Monte Carlo integration, the prediction made with model M needs to be converted into a gaussian process. In this plugin, this is done by introducing a Dropout operation after each activation layer found in the deep learning model M [22]. Such introduction of Dropout operations are handled automatically by this plugin as follows: 1) Dropout layers are enabled in evaluation time for models that already have Dropout layers inside their architecture, or 2) Dropout layers are introduced with a default probability of 0.05 after each activation layer found in the model's module layers.

The active learning process implemented in this plugin is illustrated in Figure 1.

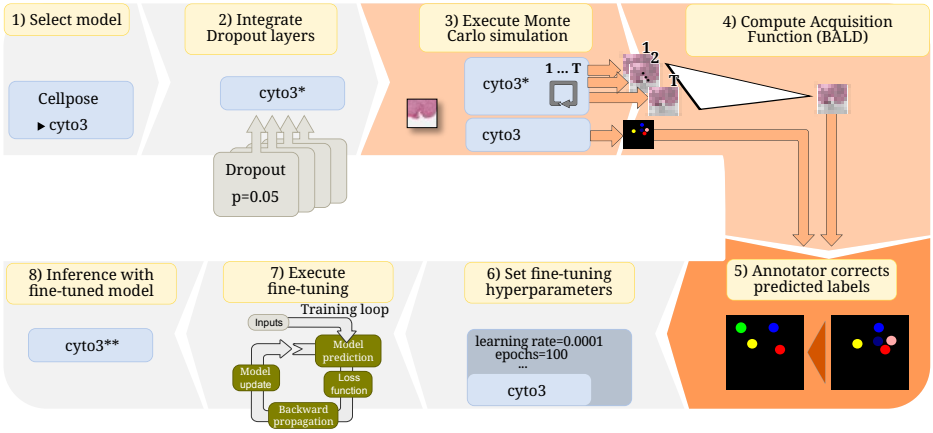


Figure 1. *Flowchart of the active learning framework implemented in the "napari-activelearning" plugin. The process is the following: 1) the user selects a model from the models registry in the plugin, 2) the plugin integrates Dropout layers into a bayesian version of the model (cyto*), 3) prediction on a sample image patch is repeated T times with the bayesian model (cyto3*) in a Monte Carlo simulation process; additionally, the baseline model (cyto3) is used to evaluate the same sample image patch once to obtain the objects labels, 4) the BALD acquisition function is computed from the results of the Monte Carlo simulation process; the resulting BALD acquisition function and objects labels are added to napari's layers list separately, 5) the annotator selects the next patch with the highest BALD score and corrects its labels using napari's builtin annotation tools, 6) the user sets up the hyper-parameters of the fine-tuning process corresponding to the baseline model (cyto3) in the plugin, 7) the wrapped fine-tuning process from the selected model is executed using the corrected labels as ground-truth, 8) the sample image patches are predicted again with the fine-tuned model (cyto3**) for comparison. Note that steps 3 and 4 are repeated for a number of times defined by the annotator to extract sufficient sample patches from the images. Additionally, the annotator can decide how many samples to correct in step 5, as well as skip labels with low BALD score.*

2.4. Implementation details

The "napari-activelearning" plugin is an open-source project implemented in the Python programming language that relies on the napari plugin architecture to offer a user-friendly interface. This plugin consists of three main component widgets that allow the execution of an end-to-end fine-tuning process from within a napari window. Such components are 1) an image groups manager widget, 2) an acquisition function configuration widget, and 3) a labels manager widget.

2.4.1. *Image Groups Manager*

This component is used to gather the metadata of the images used for training and fine-tuning into a single data structure called “image group”. The purpose of this structure, shown in Figure 2, is to define how each layer shown on the napari window will be used in the active learning workflow, such as input data, labels or annotations, and sampling masks. The gathered metadata is used to connect the input data with the selected deep learning model. This is done in this plugin by streaming the pixel data from the input image in the structure and format that the model expects it. The reformating and structuring process commonly involve reordering the spatial axes and color channels of the image data, and even casting the data into a different data type required by the model. The metadata also contains the origin of the image data, which can be a path to an image file on disk or a location in a remote storage. The origin is used by this plugin to access the pixel data directly avoiding duplicating the data when possible. Moreover, sampling masks allow the user to define the regions in the image from where the plugin can sample image patches to be used through the fine-tuning process.



Figure 2. *Image groups manager widget used to manage metadata of napari’s layers to be used for inference and fine-tuning along with the Acquisition Function Manager component. This widget is composed of four main components: a) a set of buttons to create and add image groups, layer groups, and layer channels, b) an editor that allows to define metadata related to each of the layers in the image groups, c) an editor to create sampling masks that allow to focus the active learning process on specified regions of an image defined by the user, and d) a list of the layers added to each image group along with their specific use, such as input image, labels for training, and sampling masks.*

2.4.2. *Acquisition Function Manager*

In this component, the model used for active learning can be selected from a list of registered models, and their hyper-parameters can be configured according to the user’s needs. This component uses NumPy [23] and PyTorch [24] libraries to implement the BALD [11] acquisition function and the Dropout operation insertion [22]. The outputs generated in the intermediate steps of the active learning framework are transformed using the Sci-kit Image library [25] and stored using the OME-Zarr [13] specification to make them shareable and reproducible under FAIR [26] guidelines. This component computes the acquisition function score for a set of image patches sampled from the input image defined in the Image Groups Manager component. The number of samples extracted and the total steps for the Monte Carlo integration process can be defined by the user in this widget, as shown in Figure 3. For each sampled patch, the inference and acquisition function are computed and presented to the user in a descending list on the Labels Manager widget. The “napari-activelearning” uses Cellpose [6] as default deep learning framework for inference and fine-tuning with its pre-trained models. Moreover, the code that implements this component is intended for its extension to other deep learning models with PyTorch backend [24]. This can be achieved through the inheritance of a segmentation method class (`TunableMethod`) that is used to execute pure inference, probability computations, and even fine-tuning of model weights. As an example, the integration of Cellpose models into the napari-activelearning plugin is illustrated in Figure 4. The hyper-parameters that appear at the right in Figure 3

d) had been defined in the fine-tuning function (Figure 4 c) when integrating the Cellpose model into this plugin.

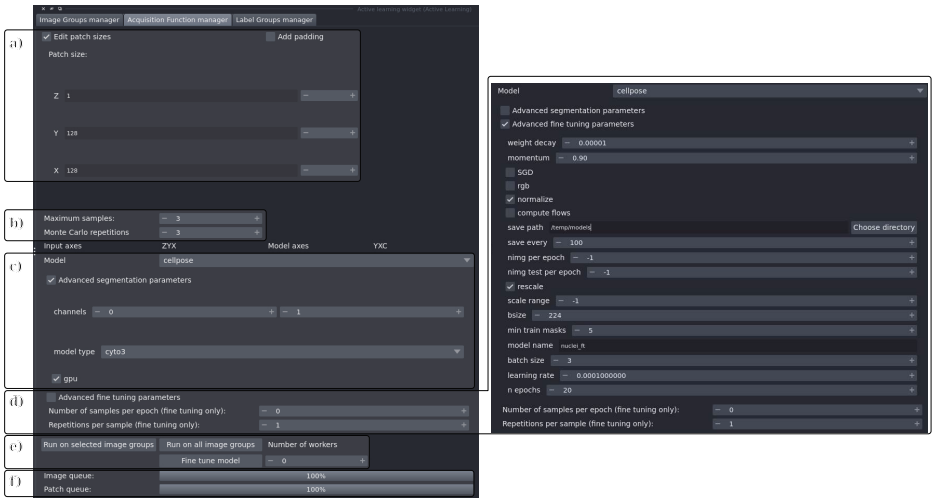


Figure 3. *Acquisition function manager widget used mainly to execute model inference and fine-tuning, along with configuration of model parameters and the active learning sampling parameters. The acquisition function manager widget is composed of six main components that allow a) select the size of the image patches extracted from the input images, b) determine the number of patches that are randomly extracted from the image and the number of repetitions in the Monte Carlo simulation for estimating the acquisition function value, c) configure the inference parameters of the selected deep learning model, d) configure the hyper-parameters for the fine-tuning process, e) execute the inference step on a single image, or all the images added to the image groups manager, as well as execute the fine-tuning process once the labels have been corrected, and f) visualize the progress of the inference step as it is applied to the images added to the image groups manager.*

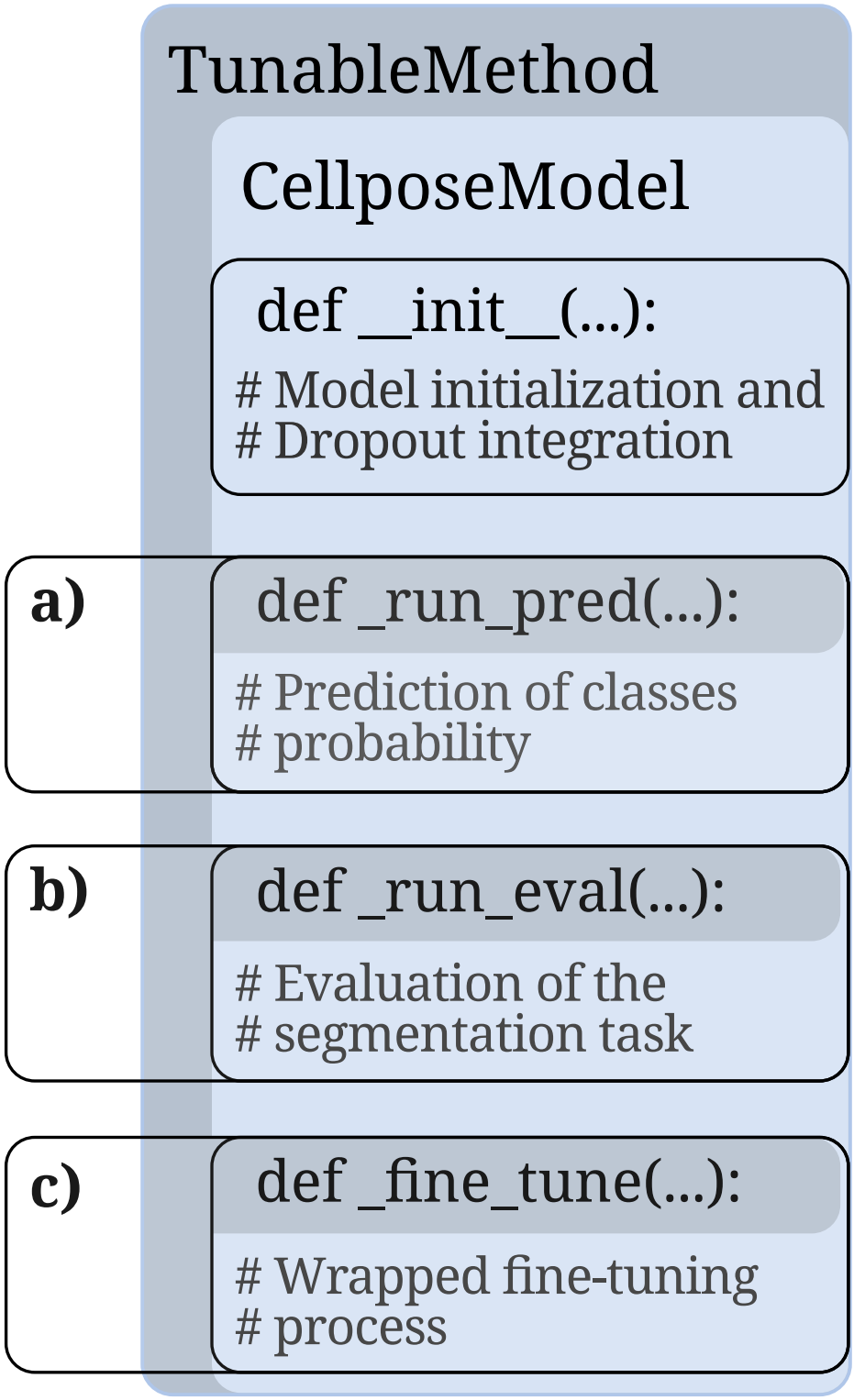


Figure 4. *Visual representation of the integration of Cellpose models as tunable methods in the napari-activelearning plugin through inheritance of the* `TunableMethod` *class. The derived class (*`CellposeModel`*) is required to implement: a) the* `_run_pred` *function that predicts the probability of each instance being assigned to the distinct domain classes; in the case of Cellpose that is the probability of each individual pixel being assigned to the foreground or background class, b) the* `_run_eval` *function that executes the*

normal evaluation process of the Cellpose model, and c) the `_fine_tune` function that wraps the existing fine-tuning process implemented in the Cellpose Python package [6].

2.4.3. *Labels Manager*

After the samples are extracted and their acquisition function scores and inferences are computed by the Acquisition Function Manager, these can be reviewed by the user. The labels predicted by the selected model can be corrected using built-in napari annotation tools. These corrected annotations can be used as new targets for fine-tuning the selected model. Additionally, this component provides a simple navigation system to move between inferred labels in space as presented in Figure 5. This system is ideal for reviewing the output generated by the selected model across the image, which can be time consuming in large-scale images, such as WSI or three-dimensional image data.

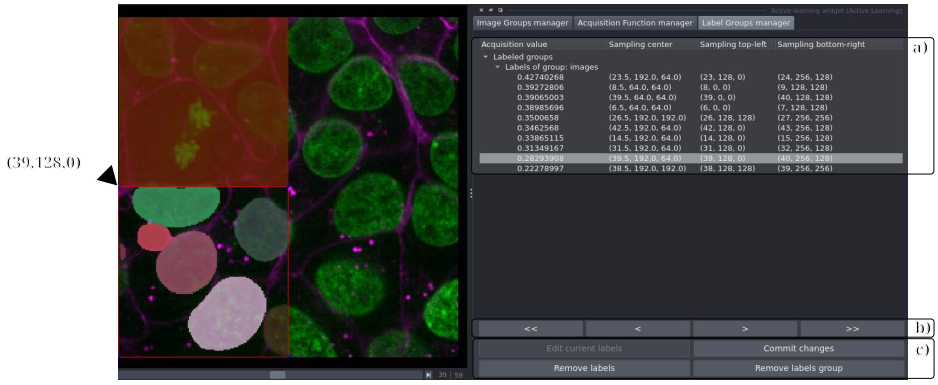


Figure 5. *Label groups manager widget used for handling sampled patches for labels correction following an active learning workflow. This widget is composed of three main components: a) the list of "label groups" that displays information of the location of the labeled image patches and their corresponding acquisition function value, b) a set of navigation buttons to move between labeled image patches, and c) a set of buttons to edit, commit changes, and remove labels and labels groups. Note that the currently selected label (at coordinates $Z = 39$, $Y = 128$, and $X = 0$ as displayed in the "Sampling top-left" column of this widget) is in edit mode, which allows to modify the labels of the objects inside the active region without altering the labels of any surrounding image patch.*

3. Results

The "napari-activelearning" source code is open source and available to the research community at the http://github.com/thejacksonlaboratory/activelearning repository. This package is distributed through PyPI (https://pypi.org/project/napari-activelearning) and can also be installed using the napari plugin manager.

3.1. Fine-tuning a Cellpose model

A comparative analysis was carried out to show the benefit of using the active learning framework implemented in this plugin for fine-tuning a deep learning model for bioimage analysis. The experiments consisted of fine-tuning the Cellpose's Cyto3 model for cell segmentation using training sets of different sizes: 5, 10, 15, 20, 25, and 30 images. The images used in the experiments are part of the Cellpose dataset (https://www.cellpose.org/dataset) [6]. This dataset was created with images from multiple domains and imaging modalities; however, only a set of 89 images for training and 11 images for testing were used in these experiments. The subset of images selected share the same imaging modality and domain, and it serves as an use case of fine-tuning a model to carry out cell segmentation on a specific imaging modality. The training sets with different sizes were created using the images with the top 5, 10, 15, 20, 25, and 30 BALD score values computed for every image

in the pool of 89 training images. The segmentation performance of the different variants of the Cyto3 model were measured in terms of the F1-score computed on the testing set of 11 images. Additionally, a paired Wilcoxon rank test was performed to determine the statistical meaningfulness of the difference between the baseline and fine-tuned models' segmentation performance. The results of the computational experiments are presented in Table 1.

Table 1. *Comparison of the segmentation performance, in terms of F1-score, between the baseline Cyto3 model from Cellpose and fine-tuned version of the same model that were trained with 5, 10, 15, 20, 25, and 30 images taken from a pool of 89 training images. The images used to fine-tune the Cyto3 model were selected based on the highest BALD score computed with this plugin.*

Model	Version	Training size	Training F1-score	Training Wilcoxon rank test (p-value)	Testing F1-score	Testing Wilcoxon rank test (p-value)
Cyto3	baseline	•	0.8561	•	0.8447	•
Cyto3	fine-tuned	5	0.8268	0.9293	0.8217	0.6036
Cyto3	fine-tuned	10	0.8838	0.0032	*0.8800*	*0.1620*
Cyto3	fine-tuned	15	0.8912	$8.18e^{-5}$	0.8785	0.2057
Cyto3	fine-tuned	20	0.8937	$3.06e^{-5}$	**0.9013**	**0.0654**
Cyto3	fine-tuned	25	0.8971	$2.17e^{-5}$	0.8854	0.1185
Cyto3	fine-tuned	30	**0.8982**	$\mathbf{1.07e^{-5}}$	0.8908	0.0741

The comparative analysis results showed the plugin implementation of the active learning framework enables improving the performance of the Cyto3 model through fine-tuning. The results show that the cell segmentation performance of the fine-tuned model can achieve a F1-score of up to 0.9013 on the selected imaging modality. This is significantly greater than the F1-score of 0.8447 obtained by the baseline Cyto3 model (p-value of 0.0654). Moreover, the computational experiments suggest that the active learning plugin can reduce the amount of samples required to be labeled in order to improve the baseline model performance. For this use case, labeling only 10 of the 89 images in the training pool is sufficient to outperform the baseline model performance in the test set (F1-score of 0.8800).

4. Conclusion

In this work, the "napari-activelearning" plugin has been introduced for carrying out fine-tuning of deep learning models for large-scale bioimage data. The napari visualization tool was used to host the plugin and serve as a graphical user interface. Additionally, by leveraging NGFF, the annotated data is stored efficiently by creating only image chunks that contain any labels information and avoiding writing empty chunks. Management of the data created through the active learning workflow is annotated following the OME-Zarr v2 specification making it ready for sharing and reproduction. Whereas this plugin was developed to facilitate adoption of deep learning models in bioimage analysis, it is not restricted to these imaging modalities and can be extended to be used with any image stored in the Zarr format. Moreover, a comparative analysis showed that this plugin can be used to improve the performance of a baseline cell segmentation model, reaching a F1-score of 0.9013 while requiring to annotate only 20 images from a pool of 89 images to achive this performance. Finally, the plugin can be used for transfer learning or simply as an interface for deep learning methods that lack graphical user interfaces for inference. A tutorial on how to use this plugin for fine-tuning a Cellpose model to carry out nuclei segmentation on the Cells 3D+2 Channels image from Sci-kit Image (included as a sample image in napari)

can be found in the supporting documents section of this paper and on the plugin's documentation website at https://thejacksonlaboratory.github.io/activelearning/tutorials.html.

References

[1] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255. doi: https://doi.org/10.1109/CVPR.2009.5206848.

[2] A. Kirillov *et al.*, "Segment Anything," in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 3992–4003. doi: https://doi.org/10.1109/ICCV51070.2023.00371.

[3] N. F. Greenwald *et al.*, "Whole-cell segmentation of tissue images with human-level performance using large-scale data annotation and deep learning," *Nat Biotechnol*, vol. 40, no. 4, pp. 555–565, 2021, doi: https://doi.org/10.1038/s41587-021-01094-0.

[4] C. Edlund *et al.*, "LIVECell—A large-scale dataset for label-free live cell segmentation," *Nature Methods*, vol. 18, no. 9, pp. 1038–1045, 2021, doi: https://doi.org/10.1038/s41592-021-01249-6.

[5] M. Y. Lee *et al.*, "CellSeg: a robust, pre-trained nucleus segmentation and pixel quantification software for highly multiplexed fluorescence images," *BMC Bioinformatics*, vol. 23, no. 1, p. 46, 2022, doi: https://doi.org/10.1186/s12859-022-04570-9.

[6] C. Stringer, T. Wang, M. Michaelos, and M. Pachitariu, "Cellpose: a generalist algorithm for cellular segmentation," *Nature Methods*, vol. 18, no. 1, pp. 100–106, 2021, doi: https://doi.org/10.1038/s41592-020-01018-x.

[7] M. Weigert and U. Schmidt, "Nuclei Instance Segmentation and Classification in Histopathology Images with Stardist," in *The IEEE International Symposium on Biomedical Imaging Challenges (ISBIC)*, 2022. doi: https://doi.org/10.1109/ISBIC56247.2022.9854534.

[8] A. Archit *et al.*, "Segment Anything for Microscopy," *Nature Methods*, vol. 22, no. 3, pp. 579–591, 2025, doi: https://doi.org/10.1038/s41592-024-02580-4.

[9] Y. Gal, R. Islam, and Z. Ghahramani, "Deep Bayesian Active Learning with Image Data." [Online]. Available: https://arxiv.org/abs/1703.02910

[10] S. Budd, E. C. Robinson, and B. Kainz, "A survey on active learning and human-in-the-loop deep learning for medical image analysis," *Medical Image Analysis*, vol. 71, p. 102062, 2021, doi: https://doi.org/10.1016/j.media.2021.102062.

[11] N. Houlsby, F. Huszár, Z. Ghahramani, and M. Lengyel, "Bayesian Active Learning for Classification and Preference Learning," *ArXiv*, 2011, doi: https://doi.org/10.48550/arXiv.1112.5745.

[12] J. Moore *et al.*, "OME-NGFF: a next-generation file format for expanding bioimaging data-access strategies," *Nature Methods*, vol. 18, no. 12, pp. 1496–1498, 2021, doi: https://doi.org/10.1038/s41592-021-01326-w.

[13] J. Moore *et al.*, "OME-Zarr: a cloud-optimized bioimaging file format with international community support," *Histochemistry and Cell Biology*, vol. 160, no. 3, pp. 223–251, 2023, doi: https://doi.org/10.1007/s00418-023-02209-1.

[14] A. Miles *et al.*, "zarr-developers/zarr-python: v2.4.0." Zenodo, 2020. doi: https://doi.org/10.5281/zenodo.3773450.

[15] J. Schindelin *et al.*, "Fiji: an open-source platform for biological-image analysis," *Nature Methods*, vol. 9, no. 7, pp. 676–682, 2012, doi: https://doi.org/10.1038/nmeth.2019.

[16] C. Pape *et al.*, "MoBIE: A Fiji plugin for sharing and exploration of multi-modal cloud-hosted big image data," *bioRxiv*, 2022, doi: https://doi.org/10.1101/2022.05.27.493763.

[17] P. Bankhead *et al.*, "QuPath: Open source software for digital pathology image analysis," *Scientific Reports*, vol. 7, no. 1, p. 16878, 2017, doi: https://doi.org/10.1038/s41598-017-17204-5.

[18] N. Sofroniew *et al.*, "napari: a multi-dimensional image viewer for Python." Zenodo, 2025. doi: https://doi.org/10.5281/zenodo.15465370.

[19] J. Maitin-Shepard *et al.*, "google/neuroglancer:." Zenodo, 2021. doi: https://doi.org/10.5281/zenodo.5573294.

[20] T. Manz *et al.*, "Viv: multiscale visualization of high-resolution multiplexed bioimaging data on the web," *Nature Methods*, 2022, doi: https://doi.org/10.1038/s41592-022-01482-7.

[21] K. M. Boergens *et al.*, "webKnossos: efficient online 3D data annotation for connectomics," *Nature Methods*, vol. 14, no. 7, pp. 691–694, 2017, doi: https://doi.org/10.1038/nmeth.4331.

[22] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning." [Online]. Available: https://arxiv.org/abs/1506.02142

[23] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: https://doi.org/10.1038/s41586-020-2649-2.

[24] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” [Online]. Available: https://arxiv.org/abs/1912.01703

[25] S. van der Walt *et al.*, “scikit-image: image processing in Python,” *PeerJ*, vol. 2, p. e453, 2014, doi: https://doi.org/10.7717/peerj.453.

[26] M. D. Wilkinson *et al.*, “The FAIR Guiding Principles for scientific data management and stewardship,” *Scientific Data*, vol. 3, no. 1, p. 160018, 2016, doi: https://doi.org/10.1038/sdata.2016.18.